

Texto Complementar - Aula 4

Git, organização de projeto e documentação mínima

1. Por que versionar o projeto?

Versionar é registrar a evolução do código. Em vez de salvar várias cópias como projeto_final, projeto_final_agora_vai ou projeto_final_corrigido, o Git permite guardar checkpoints com mensagens que explicam o que mudou.

Em sistemas reais, o histórico é tão importante quanto o código atual, porque ajuda a entender decisões, desfazer mudanças ruins e trabalhar em equipe.

2. Conceitos essenciais

Conceito	Explicação curta
Repositório	Pasta controlada pelo Git, com histórico de alterações.
Working tree	Arquivos atuais do projeto, ainda em edição.
Staging	Área onde você escolhe o que entrará no próximo commit.
Commit	Registro de uma alteração concluída e explicada por uma mensagem.
Remoto	Cópia online do repositório, geralmente no GitHub.
Push	Envio dos commits locais para o repositório remoto.

3. Organização mínima em Java Desktop

Nesta etapa, a aplicação ainda é pequena, mas já precisa de uma estrutura previsível. A organização por pacotes não é burocracia: ela reduz confusão quando o sistema cresce.

Pacote	Responsabilidade
model	Representa os dados principais do sistema, como Aluno.
service	Guarda regras e operações, como cadastrar, listar, buscar e contar.
view	Mostra a tela Swing e reage aos cliques do usuário.
app	Contém a classe Principal, responsável por iniciar o sistema.

4. Como escrever bons commits

Um bom commit deve ser pequeno, testável e fácil de explicar. A mensagem deve indicar a mudança realizada, não apenas dizer “alterações” ou “pronto”.

Mensagem fraca	Mensagem melhor
arrumei	corrige validação de semestre

coisas da aula	cria estrutura inicial do projeto
teste	adiciona botão para contar alunos
final	documenta execução no README

5. README e .gitignore

O README é a porta de entrada do projeto. Ele deve explicar objetivo, execução e organização. O .gitignore evita que arquivos gerados pelo ambiente entrem no repositório.

Regra prática: versionar código-fonte e documentação; ignorar arquivos compilados e temporários.

6. O que explicar na entrega

- Qual era o estado inicial do projeto.
- Quais commits foram feitos e o que cada um representa.
- Onde está a classe de domínio, onde está o service, onde está a tela e onde o programa inicia.
- Como a melhoria implementada passa da tela para a regra da aplicação.
- Como outra pessoa executaria o projeto a partir do README.